

Seguridad y resiliencia

Seguridad y resiliencia

Glowing Beauty S.R.L. — Automatización de recepción de facturas Departamento de Compras Septiembre 2026

1. Propósito y postura

Este documento describe cómo el sistema protege la información que maneja y cómo se comporta cuando algo falla.

La postura de fondo es que **un proceso que aprueba pagos no puede fallar hacia la aprobación**. Todo el diseño está orientado a que cualquier desvío —un error de lectura, una caída de API, un dato que no cierra— termine en un rechazo registrado o en una derivación a una persona, nunca en una aprobación silenciosa.

Los modos de falla que se describen en el punto 5 no son hipótesis: **los ocho se detectaron durante el desarrollo y el test de estrés**, y están documentados con su síntoma, su causa y la corrección aplicada. Los que siguen abiertos están identificados como tales en el punto 8.

El test de estrés se ejecutó sobre cinco presentaciones —una correcta, tres que activan códigos de rechazo distintos y una que supera el tope de aprobación automática— y todas terminaron donde correspondía.

2. Minimización de datos

2.1 Qué entra y qué se guarda

El sistema recibe comprobantes fiscales, que contienen datos de terceros: CUIT, domicilios, condición frente al IVA, detalle de productos y precios. De todo eso, la base guarda únicamente lo que el proceso de aprobación necesita.

Dato que entra	¿Se persiste?	Criterio
Importes (neto, IVA, total)	Sí	Determinan la aprobación y el monto a pagar
Fecha de emisión	Sí	Base del cálculo del plazo
CUIT del emisor	Sí	Identifica al proveedor en el legajo
Número de comprobante	Sí	Clave de la presentación
Correo remitente	Sí	Destino de la notificación
Texto completo del comprobante	No	Insumo de la lectura, no dato del proceso
Domicilio, ingresos brutos, CAE	No	No intervienen en ninguna decisión del flujo
Detalle de productos y precios unitarios	No	Información comercial ajena al circuito de aprobación
Archivo PDF original	No	Permanece en la casilla de correo

El texto completo del comprobante circula entre nodos como campo `texto_pdf` y muere al terminar la ejecución. Es el campo más voluminoso del flujo y el que más datos de terceros transporta; no escribirlo en la base es la decisión de minimización más relevante del sistema.

El PDF original no se sube a Airtable. Queda en la casilla de Compras, que es el repositorio con valor documental y ya tiene su propio control de acceso. Duplicarlo en la base multiplicaría la superficie de exposición sin agregar capacidad al proceso.

2.2 Lo que se envía al modelo

Al proveedor de IA se le manda el texto plano de un comprobante comercial. No se le envía:

- Ninguna credencial, identificador interno ni contenido de la base
- El legajo del proveedor (plazo de pago, tope de aprobación): esos datos se resuelven en Airtable y nunca entran al

prompt

- El resultado de las validaciones previas ni el historial de la factura

El modelo recibe el mínimo necesario para la única tarea que tiene: leer números de un texto. No sabe qué se hace con lo que devuelve, y esa ignorancia es deliberada: el tope de aprobación de un proveedor es información comercial sensible y no hay motivo para que salga de la organización.

2.3 Superficie de exposición

Componente	Qué expone	Control
Casilla de Compras	Comprobantes originales	Credenciales de Gmail, acceso restringido
Airtable	Registro de facturas y legajo	Cuenta con credenciales; escritura solo vía API
Vista compartida pública	Registro de facturas, en lectura	Enlace no indexado, sin permiso de edición
Telegram	Importe, proveedor y motivo de derivación	Canal privado del bot
OpenRouter	Texto del comprobante	Solo durante la llamada

La vista compartida del dashboard es el punto de exposición deliberada: es pública por requisito del trabajo. Expone datos de una empresa ficticia con proveedores ficticios. **En una implementación real, el dashboard no se publicaría como vista abierta:** Airtable ofrece restricción por contraseña y por dominio de correo en sus planes pagos, y la interfaz de control se serviría por ahí.

3. Credenciales y secretos

Secreto	Dónde vive	Exposición
API key de OpenRouter	Credencial de n8n	No aparece en ningún nodo ni en el JSON exportado
Token del bot de Telegram	Credencial de n8n	Ídem
OAuth de Gmail	Credencial de n8n	Ídem
API key de Airtable	Credencial de n8n	Ídem

Las cuatro se almacenan en el gestor de credenciales de n8n y se referencian por nombre. **El JSON exportado del workflow no contiene ningún secreto:** conserva la referencia a la credencial, no su valor. Esto permite publicar el flujo completo en un repositorio abierto sin filtrar nada.

En el video de demostración, las pantallas de configuración de credenciales se omiten o se cubren.

Valores de entorno. El identificador del chat de Telegram que recibe las solicitudes de aprobación no está escrito dentro del nodo: se inyecta en un nodo `Config` al inicio del flujo y los nodos que lo necesitan lo referencian por expresión. No es un secreto —no da acceso a nada sin el token del bot— pero es un valor que cambia entre entornos, y concentrarlo en un solo lugar evita tener que buscarlo nodo por nodo. Queda además visible en el canvas, que es preferible a una variable oculta en la configuración de la cuenta.

4. Manejo de errores

4.1 Estrategia

El error handling está resuelto con **salidas de error por nodo**, no con un workflow de error global. La diferencia es que cada falla puede tener una reacción distinta y queda registrada con el contexto de dónde ocurrió.

Tres nodos tienen activado *On Error: Continue (using error output)* junto con *Retry On Fail*:

Nodo	Falla que cubre	Reacción
Extraer datos IA	API caída, límite de peticiones, modelo retirado	Reintento automático; si persiste, registro en <code>Log_Errores</code>
Guardar Aprobada	Rechazo de escritura de Airtable	Ídem

Guardar en revisión	Rechazo de escritura de Airtable	Ídem
---------------------	----------------------------------	------

El reintento cubre la falla transitoria, que es la mayoría. La salida de error cubre la persistente: en vez de cortar la ejecución con un error opaco, el flujo continúa hacia un nodo que escribe qué pasó y dónde.

4.2 El registro de errores

Los nodos de log escriben en `Log_Errores` el workflow, el nodo que falló y el mensaje devuelto, con una expresión que degrada de forma ordenada:

```
{{ $json.error?.message || JSON.stringify($json.error) || 'Error sin detalle' }}
```

Los tres niveles existen porque la forma del objeto de error varía según el origen de la falla. Un log que dice "Error sin detalle" es pobre, pero es infinitamente mejor que una ejecución que falla sin dejar rastro.

Evidencia de funcionamiento. La tabla `Log_Errores` contiene dos registros reales del nodo `Guardar Aprobada`, generados durante el desarrollo cuando una factura sin proveedor vinculado llegaba a la rama de aprobación y Airtable rechazaba la escritura. Se conservan deliberadamente: son la prueba de que el mecanismo captura fallas reales y no solo fallas simuladas.

La ruta de error de la extracción con IA no llegó a dispararse en una ejecución real. Está configurada igual que las otras dos.

4.3 Elección de estrategia por nodo

No todos los nodos llevan salida de error, y la ausencia es intencional:

- **Los nodos de código no fallan por causas externas.** Sus errores serían de programación, y el lugar de corregirlos es el código, no un manejador.
- **El trigger no puede tener salida de error:** si Gmail no responde, no hay ejecución que capturar.
- **Los nodos de notificación por correo** pueden fallar sin comprometer el dato: la factura ya quedó registrada con su estado. Un correo no enviado es un problema de comunicación, no de integridad.

El criterio es proteger la ruta que produce o persiste información, no cada paso por igual.

5. Modos de falla detectados y corregidos

Esta sección documenta fallas reales encontradas durante el desarrollo. Cada una está descrita con su síntoma, su causa y la corrección aplicada, porque el valor está en el patrón, no en el incidente.

5.1 Pisado del estado de rechazo entre nodos

Síntoma. Facturas con un rechazo detectado en una validación temprana llegaban al final del flujo como si estuvieran limpias, y terminaban aprobadas o cayendo en la salida de descarte del ruteo.

Causa. Cada nodo de código inicializaba su propia variable `codigo` en `null` y cerraba con `rechazado: codigo !== null`. Un nodo que no detectaba nada por su cuenta escribía `false` sobre el `true` que venía del nodo anterior. El sistema perdía el motivo de rechazo en el camino.

Corrección. Los tres nodos afectados pasaron a consultar el estado entrante antes de escribir el suyo:

```
rechazado: ctx.json.rechazado === true || codigo !== null,
codigo_rechazo: ctx.json.codigo_rechazo || codigo,
motivo_rechazo: ctx.json.motivo_rechazo || motivo,
```

En el nodo de verificación se agregó además una constante `rechazadoFinal` usada también en la condición de derivación, para que una factura ya rechazada no se marque a la vez como pendiente de revisión y genere una solicitud de aprobación sobre algo que ya está descartado.

El patrón. En un flujo donde el estado se acumula nodo a nodo, cada escritura de un campo de control es una oportunidad de borrar lo que otro nodo decidió. La regla que quedó: **un campo de control se escribe consultando su valor anterior, nunca desde cero**. Es el mismo principio que hace peligroso un Update de Airtable con campos vacíos en el mapeo, descrito en el manual de datos.

Gravedad. Alta: es el único fallo encontrado que podía producir una aprobación indebida.

5.2 Corte en silencio por falta de resultados

Síntoma. Presentaciones con un CUIT fuera del legajo terminaban sin registro y sin error. La ejecución figuraba como exitosa y no había hecho nada.

Causa. El nodo de búsqueda en Airtable devolvía cero elementos cuando el CUIT no existía, y no detiene una rama sin elementos sin señalarlo como falla.

Corrección. Se activó *Always Output Data* en el nodo de búsqueda. Ahora devuelve un elemento vacío, la verificación de existencia de proveedor da falso, y el rechazo RC-01 se dispara como corresponde.

El patrón. Una ejecución marcada como exitosa no garantiza que haya hecho algo. Los nodos de búsqueda son el punto donde un flujo puede terminar en silencio, y el caso "no encontrado" es exactamente el que suele necesitar una reacción.

5.3 Recorrido completo de una factura ya rechazada

Síntoma. Una presentación sin adjunto hacía fallar el nodo de extracción de PDF. Presentaciones ya rechazadas consumían igual la llamada al modelo.

Causa. No había corte entre la detección del rechazo y el resto de la cascada. El flujo detectaba el problema y seguía ejecutando todos los pasos posteriores.

Corrección. Se agregó un nodo condicional entre el armado de contexto y la extracción del PDF, que desvía las presentaciones ya rechazadas directo al registro de rechazo.

Impacto medido. La presentación sin adjunto pasó a resolverse en **12 segundos**, contra los 2 a 4 minutos de las que atraviesan la extracción con IA. El corte no solo evita un error: hace cierta la validación en cascada que el documento de arquitectura describe.

5.4 Cambio en el contrato de la respuesta del modelo

Síntoma. Tras cambiar de modelo, **todas** las facturas empezaron a derivarse a supervisión humana por confianza baja, incluidas las correctas.

Causa. El modelo nuevo devolvía el campo de confianza como `confidence`, en inglés. El nodo de verificación lo leía como `confianza`, obtenía indefinido, lo convertía a cero, y cero está por debajo del umbral de 0,70.

Corrección. `Number(ia.confianza ?? ia.confidence) || 0`, que acepta los dos nombres.

El patrón. El contrato con un modelo de lenguaje es el único del sistema que no controla la organización, y cambia cuando cambia el modelo. Conviene leerlo de forma tolerante y validar la forma de la respuesta antes de confiar en ella. Es también la razón por la que el prompt especifica el esquema de salida de forma explícita.

Nota sobre la dirección del fallo. El error derivó facturas correctas a revisión humana, que es el sentido correcto de la falla: molesta a una persona, no aprueba de más. Si el campo faltante hubiera sido el tope o el resultado de la verificación aritmética, la consecuencia habría sido la opuesta.

5.5 Indisponibilidad y rotación forzada del modelo

Síntoma. Siete modelos gratuitos distintos quedaron inutilizables durante el desarrollo, por cuatro causas: límite de peticiones persistente, retiro del catálogo gratuito, paso a modalidad paga, y respuestas inservibles para la tarea.

Corrección. Sustitución del modelo, que en este diseño es el valor de un campo. El flujo no requirió cambios estructurales en ninguna de las siete rotaciones.

El patrón. La disponibilidad de un proveedor de IA es una dependencia externa que puede desaparecer sin aviso y no figura en ninguna tabla de precios. Las mitigaciones que el diseño ya incorpora:

- **Integración contra una pasarela, no contra un proveedor.** OpenRouter expone varios catálogos con una credencial; el modelo es un parámetro. Es lo que hizo posible siete sustituciones sin tocar el flujo.
- **Prompt independiente del modelo.** No usa sintaxis ni capacidades propietarias.
- **Parseo tolerante.** Cada modelo escribe el JSON a su manera; el nodo de verificación aguanta las variantes conocidas (ver 5.7).

La mitigación operativa, en producción, es contratar un tier pago: el consumo proyectado del sistema es de centavos mensuales y la diferencia se paga sola con evitar una interrupción.

Cuando el modelo falla de verdad —no que responda distinto, sino que no responda— actúa el reintento del nodo y, si persiste, la salida de error hacia el registro. La factura no se pierde: queda el rastro de que entró y de que la lectura falló. La tabla `Log_Errores` contiene la evidencia de estas fallas reales, con los mensajes devueltos por la pasarela.

5.6 Reprocesamiento de comprobantes

Síntoma. Una consulta del trigger levantó dos correos ya procesados. Generó un registro incompleto en la base y una solicitud de aprobación humana por una factura que ya estaba resuelta.

Causa. El trigger consulta los correos no leídos de la casilla y **no los marca como leídos** al procesarlos. Un comprobante ya procesado vuelve a ser candidato en consultas posteriores, y cada relectura genera una ejecución nueva y un registro duplicado.

Estado: mitigado de forma operativa, no resuelto en el flujo. La versión del nodo utilizada no ofrece la opción de marcar como leído: el único ajuste disponible es el prefijo de los adjuntos. La mitigación actual es marcar cada correo como leído después de procesarlo y no republicar el workflow mientras haya correos pendientes sin leer.

Resolución prevista. Marcar el correo como leído inmediatamente después de la lectura, o registrar el identificador del mensaje en la base y descartar los ya vistos. La segunda alternativa cubre además el control de presentaciones duplicadas, que quedó fuera del alcance de esta entrega. Está desarrollada en el manual de datos.

Gravedad. Media. No produce una aprobación indebida —la factura vuelve a recorrer las mismas validaciones y llega al mismo resultado— pero ensucia el registro con duplicados y genera solicitudes de aprobación espurias, que a la larga es el peor efecto: una persona que recibe pedidos redundantes deja de leerlos con atención.

5.7 Respuestas mal formadas del modelo

Síntoma. Facturas correctas derivadas a supervisión humana con el motivo "la IA devolvió una respuesta no parseable".

Causa. Cada modelo escribe el JSON a su manera, y varios lo escriben mal. Tres defectos distintos aparecieron con tres modelos distintos: cadena de razonamiento antepuesta al objeto, marcas de bloque de código envolviéndolo, y una comilla de cierre en números que nunca abrieron comillas (`"neto":98920.00"` ,).

Corrección. El nodo de verificación limpia la respuesta antes de parsearla: elimina marcas de código, recorta desde la primera llave de apertura hasta la última de cierre, y suprime la comilla que sigue a un número no entrecomillado. El detalle está en el manual operativo de datos.

El patrón. El contrato con un modelo de lenguaje es el único del sistema que la organización no controla, y no se cumple de forma confiable. Conviene leerlo de forma defensiva y no asumir que una respuesta sintácticamente correcta es lo normal. El recorte por llaves es la defensa más general, porque no depende de qué escriba el modelo alrededor del objeto.

Dirección del fallo. Los tres defectos derivaron facturas correctas a revisión humana, que es el sentido seguro: molestan a una persona, no aprueban de más.

5.8 Respuesta vacía por agotamiento del presupuesto de tokens

Síntoma. El nodo del modelo devuelve una cadena vacía. La ejecución figura como exitosa, no se dispara ninguna ruta de error, y la factura termina en supervisión humana.

Causa. El modelo consumió el límite de tokens de salida razonando internamente y no llegó a emitir el JSON. Los tokens de razonamiento cuentan contra el tope pero no aparecen en el campo de texto de la respuesta. La metadata lo confirma: `finish_reason: "length"` con el contador de salida exactamente en el tope configurado.

Corrección. Elevar el tope a un valor que cubra el razonamiento interno más la respuesta útil.

El patrón, y por qué importa para la seguridad. Es la falla más engañosa que apareció: **no hay error, no hay excepción, y el sistema la reporta como éxito**. Una llamada que agota su presupuesto sin producir salida es indistinguible, desde el flujo, de una que respondió algo vacío por otra razón. Sin inspeccionar `finish_reason` el diagnóstico es imposible.

Dos lecciones de diseño:

- **Un límite mal dimensionado es un modo de falla, no una optimización.** El tope se fijó pensando en el tamaño de la respuesta esperada, que es la intuición equivocada.
- **El éxito reportado por una integración no garantiza que haya producido algo útil.** Vale para este caso y para el corte en silencio del nodo de búsqueda descrito en 5.2: son la misma clase de problema.

Deuda asociada. Hoy una respuesta vacía cae al bloque `catch` y termina en supervisión humana. Corresponde que además quede registrada en `Log_Errores`, porque es un fallo de la integración y no una duda sobre el contenido de la factura. Está en el punto 8.

6. Supervisión humana

6.1 Cuándo interviene una persona

La automatización no decide sola en tres situaciones:

Disparador	Por qué
Monto superior al tope del proveedor	Umbral de riesgo económico definido por el negocio, por proveedor
Confianza de lectura menor a 0,70	El sistema no confía en su propia lectura
Discrepancia detectada	Los datos no cierran y el caso requiere criterio

Los tres desembocan en la misma solicitud por Telegram, con el detalle de la factura y el motivo explícito de la derivación.

El tope por proveedor es el control de daño económico del sistema. Una aprobación automática nunca puede superar un monto que el negocio definió por escrito en el legajo. Es el límite que hace tolerable el resto del diseño: aun en el peor caso de un error de lectura no detectado, el monto aprobado sin intervención humana tiene techo.

6.2 Trazabilidad de la decisión

Cada decisión humana deja registro: quién aprobó, cuándo, y el comentario libre de quien decidió. La factura pasa por tres estados verificables —derivada, respondida, resuelta— y todos quedan en la base.

Limitación conocida. El campo de aprobador se completa con un valor fijo que identifica el canal, no a la persona: el formulario de Telegram no devuelve la identidad de quien responde. Cualquiera con acceso al chat puede aprobar, y el registro no distingue entre ellos.

En producción esto se resuelve con un canal por usuario o verificando el identificador de quien responde contra una lista de aprobadores autorizados. Para un circuito de aprobación de pagos, la identificación del aprobador no es opcional, y se declara acá como deuda explícita.

6.3 Qué pasa si nadie responde

La solicitud tiene un plazo de espera de 45 minutos. Vencido, la ejecución continúa por una salida de descarte que marca la factura como pendiente y deja constancia de la falta de respuesta.

Sin ese camino, una ejecución sin respuesta quedaría pausada de forma indefinida, consumiendo una ejecución del plan y sin dejar rastro de por qué. **El diseño trata la no respuesta como un resultado posible, no como un estado imposible.** La factura no se aprueba ni se rechaza por omisión: queda visible como pendiente en el dashboard.

7. Integridad del flujo

7.1 Ausencia de bucles

El flujo **no contiene bucles ni nodos de iteración**. Cada ejecución procesa un correo, recorre una cascada lineal de validaciones, toma una rama y termina. No hay reentrada, ni nodo que se llame a sí mismo, ni condición que pueda repetirse sin avanzar.

La única espera del flujo es la supervisión humana, y está acotada por el plazo de 45 minutos descrito en 6.3.

El único mecanismo de repetición es el reintento automático de los nodos con salida de error, que está limitado por la configuración del propio nodo y no puede prolongarse de forma indefinida.

7.2 Validación de la entrada

Todo lo que entra al sistema viene de afuera, y se trata como no confiable hasta que pasa la cascada:

Control	Efecto
Patrón exacto del asunto	Un asunto que no cumple el formato no produce ni CUIT ni número de comprobante
Tipo de archivo	Solo PDF, verificado por tipo declarado y por extensión
Tamaño máximo	5 MB, antes de procesar el contenido
Texto seleccionable	Descarta imágenes y escaneos
CUIT contra el legajo	Solo se procesan comprobantes de proveedores registrados
Cotejo de CUIT	El CUIT del asunto debe coincidir con el impreso en el comprobante

El último control es específicamente antifraude: impide que el comprobante de un proveedor se presente bajo la identidad de otro. Un remitente cualquiera puede mandar un correo a la casilla; lo que no puede es hacer que el sistema atribuya una factura a un legajo que no le corresponde.

Restringir la entrada a PDF con texto seleccionable también es una decisión de seguridad, no solo de costos. El modelo recibe texto plano extraído de un documento validado, no un archivo arbitrario, y el camino de rechazo es determinístico en vez de depender de que el modelo interprete bien una imagen.

8. Deuda identificada

Lo que sigue abierto, con su criticidad. Esta sección existe porque un riesgo documentado es administrable y uno que nadie anotó, no.

#	Punto	Criticidad	Resolución
1	El aprobador no se identifica individualmente	Alta	Canal por usuario o lista blanca de identificadores de Telegram
2	Reprocesamiento de correos no leídos	Media	Persistir el identificador del mensaje y descartar los ya vistos
3	Log_Errores no registra el momento del error	Media	Campo de hora de creación, que Airtable completa de forma retroactiva
4	Una respuesta vacía del modelo termina en supervisión humana en vez de en el registro de errores	Media	Detectar cadena vacía antes del parseo y derivarla a la ruta de error
5	Dashboard publicado como vista abierta	Baja en este contexto	En producción, restricción por contraseña o por dominio
6	Sin control de presentaciones duplicadas	Baja	Mismo mecanismo que el punto 2

El punto 1 es el único que impide un uso real del sistema. Los puntos 2, 3 y 4 degradan la capacidad de auditar y diagnosticar sin comprometer la integridad de los datos. Los dos últimos son ajustes de entorno.

Los puntos 3 y 4 comparten una raíz: **el sistema registra bien los fallos que se presentan como error, y mal los que se presentan como éxito**. Una llamada que devuelve vacío y una escritura que no ocurre pasan desapercibidas porque nada las señala.

9. Resumen

Requisito	Cómo se cumple
Minimización de datos	Se persiste lo que el proceso necesita; el texto del comprobante, el detalle comercial y el archivo original no se guardan en la base
Protección de credenciales	Las cuatro credenciales viven en el gestor de n8n; el flujo exportado no contiene secretos
Manejo de errores	Salidas de error con reintento en los tres nodos que producen o persisten datos, con registro en tabla

	dedicada
Registro de fallas de la IA	Nodo de log dedicado en la salida de error de la extracción
Supervisión humana	Tres disparadores, decisión con comentario, trazabilidad de estado y plazo de espera acotado
Control de bucles	El flujo es lineal, sin iteración ni reentrada
Validación de la entrada	Siete controles antes de que el contenido llegue al modelo
Sin datos incrustados	Los valores de entorno viven en un nodo <code>Config</code> y se referencian por expresión
Tolerancia a fallas del modelo	Parseo defensivo ante tres defectos de formato observados, y sustitución del modelo sin cambios estructurales

El criterio que ordena todo el diseño: el sistema falla hacia el rechazo o hacia la persona, nunca hacia la aprobación. Los modos de falla que se encontraron durante el desarrollo se comportaron así en todos los casos menos uno —el pisado del estado de rechazo, descrito en 5.1— y ese se corrigió.